

3129. Find All Possible Stable Binary Arrays I Solve

Medium

 Topics

 Companies

 Hint

You are given 3 positive integers `zero`, `one`, and `limit`.

A [binary array](#) `arr` is called **stable** if:

- The number of occurrences of 0 in `arr` is **exactly** `zero`.
- The number of occurrences of 1 in `arr` is **exactly** `one`.
- Each [subarray](#) of `arr` with a size greater than `limit` must contain **both** 0 and 1.

Return the *total* number of **stable** binary arrays.

Since the answer may be very large, return it **modulo** `$10^9 + 7$` .

Example 1:

Input: zero = 1, one = 1, limit = 2

Output: 2

Explanation:

The two possible stable binary arrays are `[1,0]` and `[0,1]`, as both arrays have a single 0 and a single 1, and no subarray has a length greater than 2.

Example 2:

Input: zero = 1, one = 2, limit = 1

Output: 1

Explanation:

The only possible stable binary array is `[1,0,1]`.

Note that the binary arrays `[1,1,0]` and `[0,1,1]` have subarrays of length 2 with identical elements, hence, they are not stable.

Example 3:

Input: zero = 3, one = 3, limit = 2

Output: 14

Explanation:

All the possible stable binary arrays are `[0,0,1,0,1,1]`, `[0,0,1,1,0,1]`, `[0,1,0,0,1,1]`, `[0,1,0,1,0,1]`, `[0,1,0,1,1,0]`, `[0,1,1,0,0,1]`, `[0,1,1,0,1,0]`, `[1,0,0,1,0,1]`, `[1,0,0,1,1,0]`, `[1,0,1,0,0,1]`, `[1,0,1,0,1,0]`, `[1,0,1,1,0,0]`, `[1,1,0,0,1,0]`, and `[1,1,0,1,0,0]`.

Constraints:

- `1 <= zero, one, limit <= 200`

Python:

class Solution:

def numberOfStableArrays(self, zero: int, one: int, limit: int) -> int:

mod = 10 ** 9 + 7

dp0 = [[0] * (one + 1) for _ in range(zero + 1)] # i 0s + j 1s ending with 0

dp1 = [[0] * (one + 1) for _ in range(zero + 1)] # i 0s + j 1s ending with 1

Base cases: only zeros or only ones => only one string if len <= min(limit, zero/one)

for i in range(1, min(zero, limit) + 1):

dp0[i][0] = 1

for j in range(1, min(one, limit) + 1):

dp1[0][j] = 1

DP iterations

for i in range(1, zero + 1):

for j in range(1, one + 1):

dp0[i][j] = sum(dp1[i - k][j] for k in range(1, min(limit, i) + 1)) % mod

dp1[i][j] = sum(dp0[i][j - k] for k in range(1, min(limit, j) + 1)) % mod

return (dp0[zero][one] + dp1[zero][one]) % mod

JavaScript:

```
/**
 * @param {number} zero
 * @param {number} one
 * @param {number} limit
 * @return {number}
 */
var numberOfStableArrays = function(zero, one, limit) {
    const modMe = 10 ** 9 + 7

    const rowLen = 1 + one
    const table = new Array(1 + zero)
    for (const i of table.keys()) {
        const row = table[i] = new Array(rowLen)
        for (const j of row.keys())
            row[j] = new Array(3) // 3 channels
    }

    function solve(zeroCount, oneCount, channel) {
        if (zeroCount === 0) {
            return Number(oneCount <= limit)
        }
        if (oneCount === 0) {
            return Number(zeroCount <= limit)
        }

        if (
            channel === 0 && oneCount > zeroCount * limit ||
            channel === 1 && oneCount > (1 + zeroCount) * limit ||
            channel === 1 && zeroCount > oneCount * limit ||
            channel === 0 && zeroCount > (1 + oneCount) * limit
        )
            return 0

        let needsZero = true, needsOne = true
        if (channel === 0) {
            needsOne = false
        } else if (channel === 1) {
            needsZero = false
        }

        let existing = table[zeroCount][oneCount][channel]
```

```

if (existing !== undefined) return existing

let result = 0
if (needsZero === true) {
  const ub = 1 + Math.min(limit, zeroCount)
  for (let moreZeroes = 1, zeroCountNext = zeroCount - moreZeroes;
    moreZeroes < ub; moreZeroes++, zeroCountNext--) {
    const subresult = solve(zeroCountNext, oneCount, 1)
    result = (result + subresult) % modMe
  }
}
if (needsOne === true) {
  const ub = 1 + Math.min(limit, oneCount)
  for (let moreOnes = 1, oneCountNext = oneCount - moreOnes;
    moreOnes < ub; moreOnes++, oneCountNext--) {
    const subresult = solve(zeroCount, oneCountNext, 0)
    result = (result + subresult) % modMe
  }
}

return table[zeroCount][oneCount][channel] = result
}

```

```

const result = solve(zero, one, 2)
return result
};

```

Java:

```

class Solution {
  public int numberOfStableArrays(int zero, int one, int limit) {
    final int mod = 1_000_000_007;

    int[][] dp0 = new int[zero + 1][one + 1]; // i 0s + j 1s ending with 0
    int[][] dp1 = new int[zero + 1][one + 1]; // i 0s + j 1s ending with 1

    // Base cases: only zeros or only ones => only one string if len <= min(limit, zero/one)
    for (int i = 1; i <= Math.min(zero, limit); ++i) dp0[i][0] = 1;
    for (int j = 1; j <= Math.min(one, limit); ++j) dp1[0][j] = 1;

    // DP iterations
    for (int i = 1; i <= zero; ++i) {
      for (int j = 1; j <= one; ++j) {
        for (int k = 1; k <= Math.min(limit, i); ++k)

```

```
        dp0[i][j] = (dp0[i][j] + dp1[i - k][j]) % mod;
    for (int k = 1; k <= Math.min(limit, j); ++k)
        dp1[i][j] = (dp1[i][j] + dp0[i][j - k]) % mod;
    }
}

return (dp0[zero][one] + dp1[zero][one]) % mod;
}
}
```